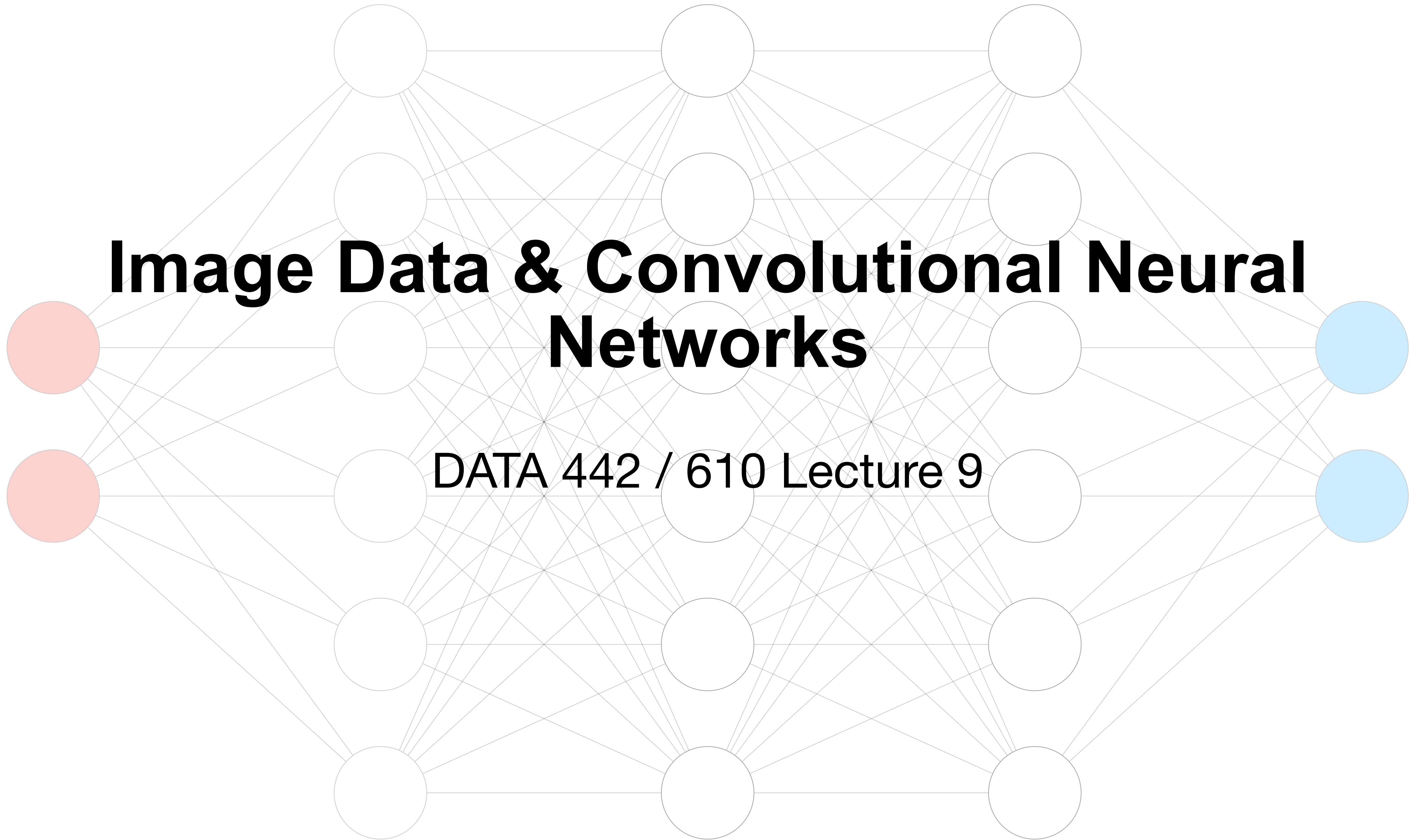


Image Data & Convolutional Neural Networks

DATA 442 / 610 Lecture 9



Outline

- Images vs. structured data
- Why CNNs?
- The discrete convolution in all its glory
- CNN demo

Relevant Reading: *Deep Learning*, Chapter 9

Structured data

Structured data follows a defined **schema**

Well-suited to classical ML

Features are precisely defined and consistent across data elements

[The Iris data set: In search of the source of virginica](#)

By A. Unwin, K. Kleinman. 2021

Published in Significance, 2021

Variables Table

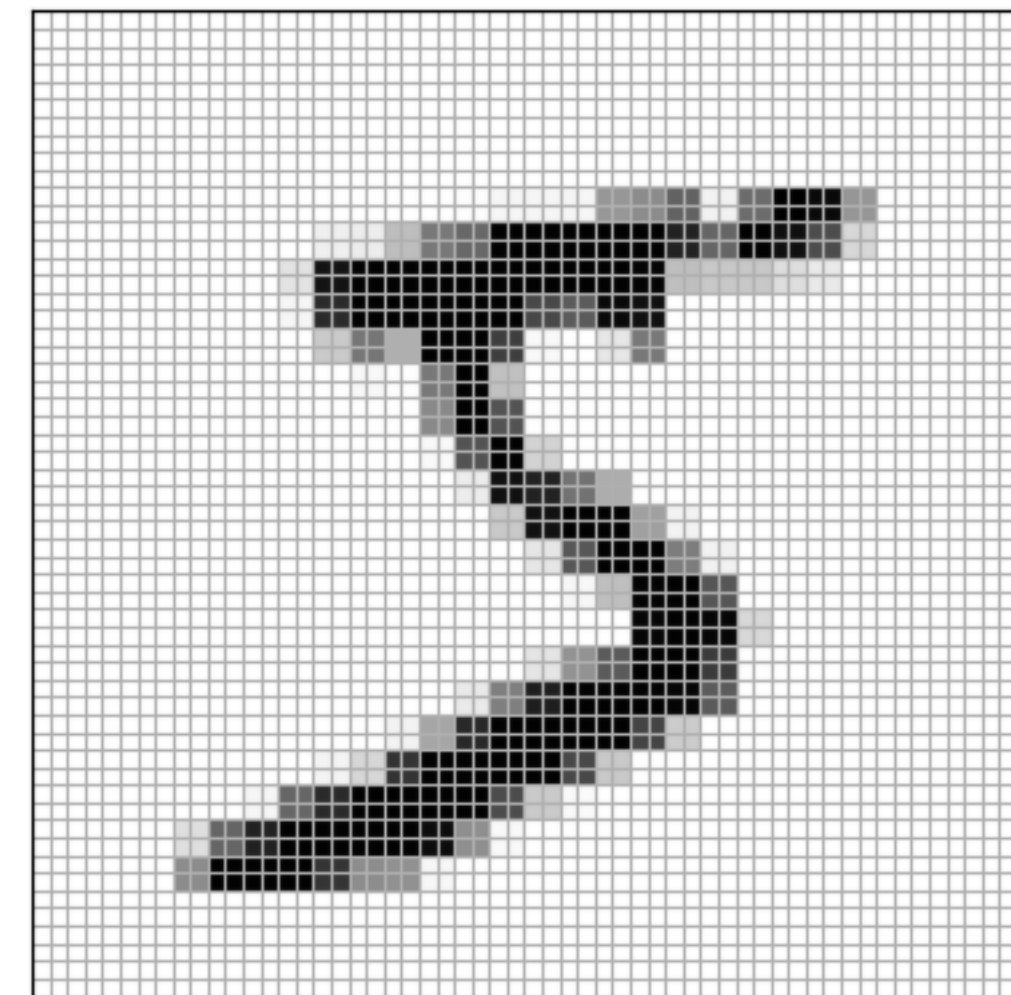
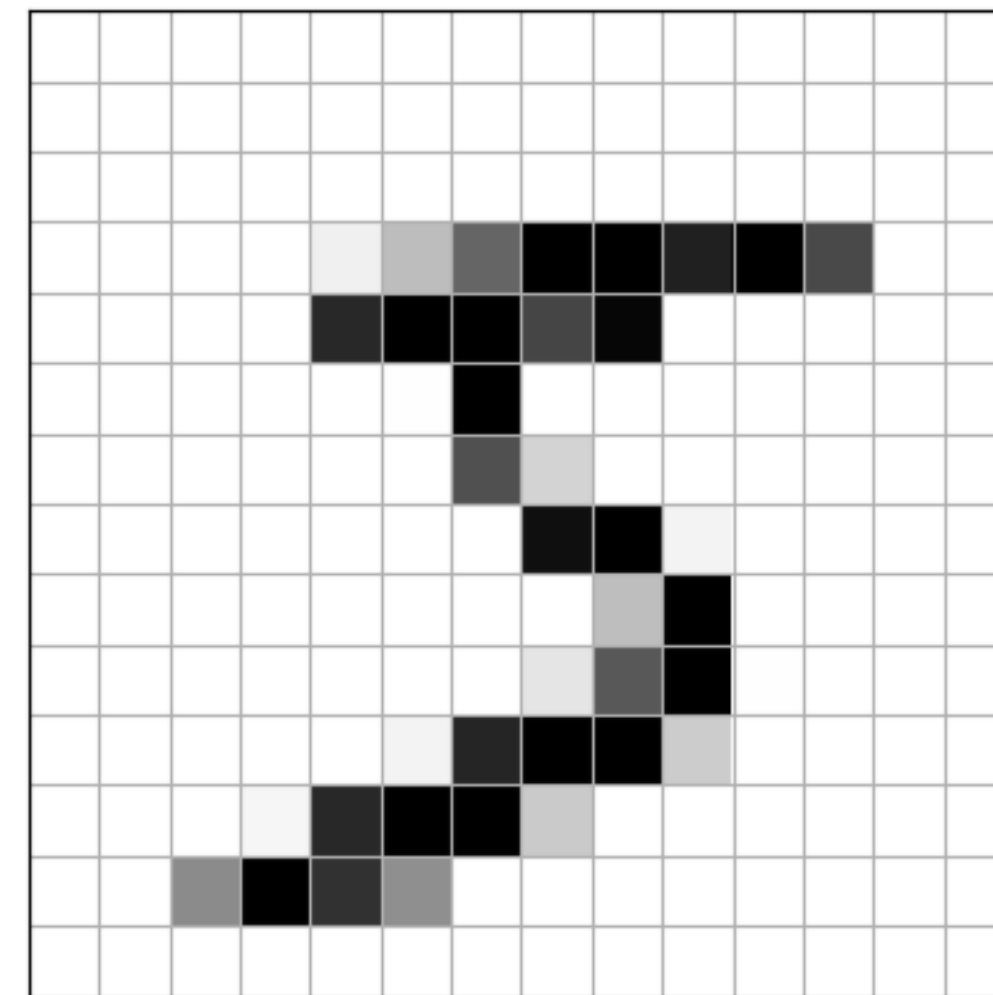
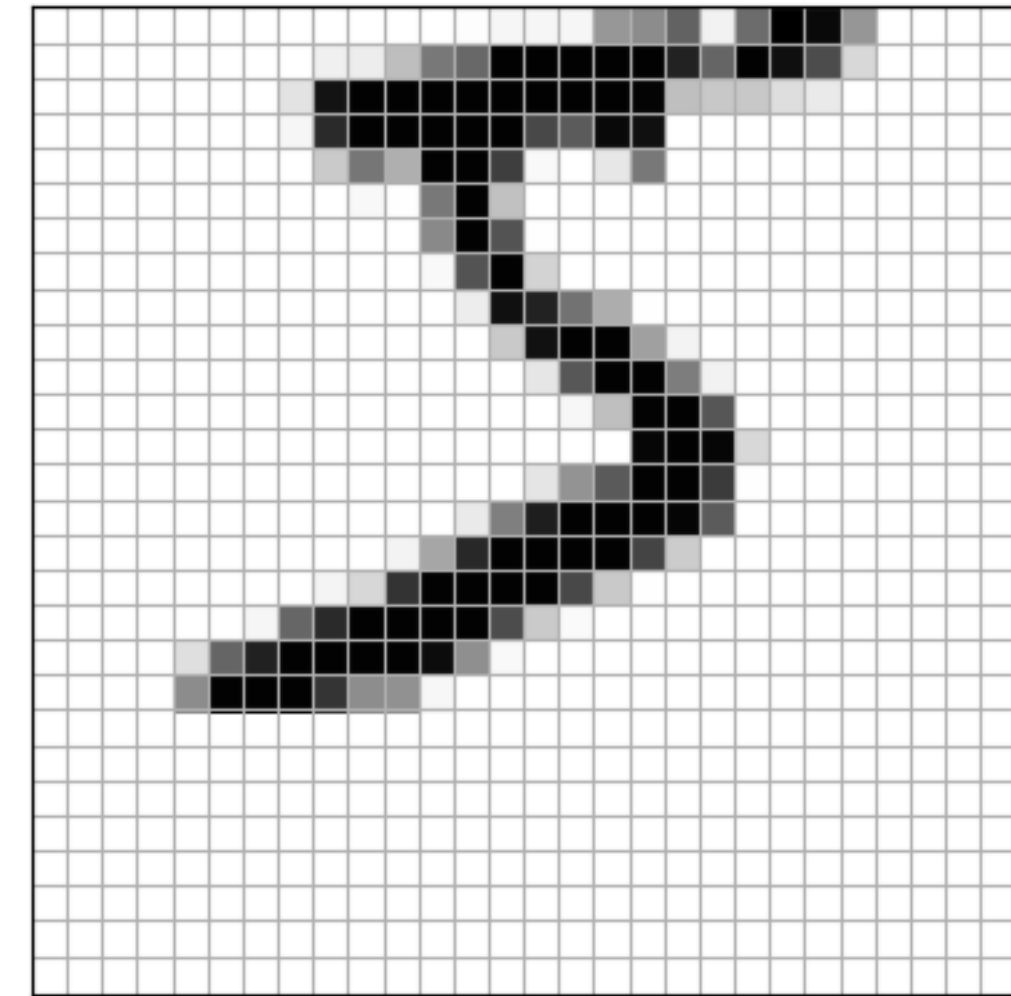
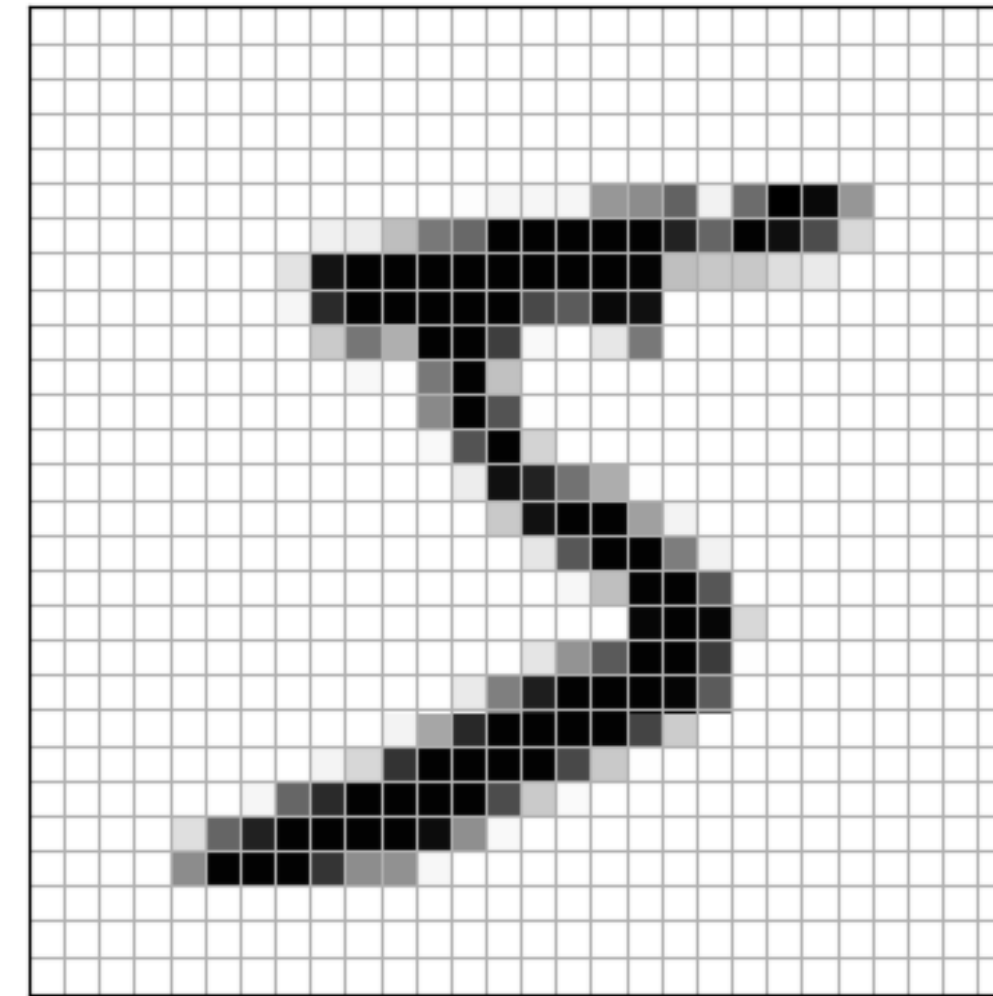
Variable Name	Role	Type	Description	Units	Missing Values
sepal length	Feature	Continuous		cm	no
sepal width	Feature	Continuous		cm	no
petal length	Feature	Continuous		cm	no
petal width	Feature	Continuous		cm	no
class	Target	Categorical	class of iris plant: Iris Setosa, Iris Versicolour, or Iris Virginica		no

Image data

Image **data** is an array of pixel values

Image **features** are encoded within pixels

Q: How can we efficiently identify features within high-dimensional unstructured image data?



Biological inspiration



J. Physiol. (1959) 148, 574–591

RECEPTIVE FIELDS OF SINGLE NEURONES IN THE CAT'S STRIATE CORTEX

BY D. H. HUBEL* AND T. N. WIESEL*

*From the Wilmer Institute, The Johns Hopkins Hospital and
University, Baltimore, Maryland, U.S.A.*

(Received 22 April 1959)

In the central nervous system the visual pathway from retina to striate cortex provides an opportunity to observe and compare single unit responses at several distinct levels. Patterns of light stimuli most effective in influencing units at one level may no longer be the most effective at the next. From differences in responses at successive stages in the pathway one may hope to gain some understanding of the part each stage plays in visual perception.

Nobel Prize in Physiology or Medicine 1981

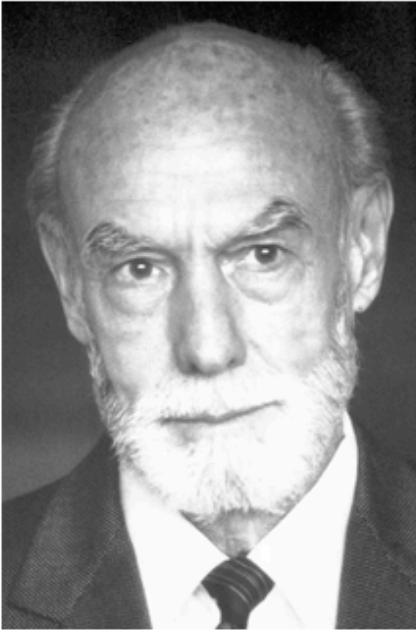


Photo from the Nobel Foundation archive.

Roger W. Sperry

Prize share: 1/2

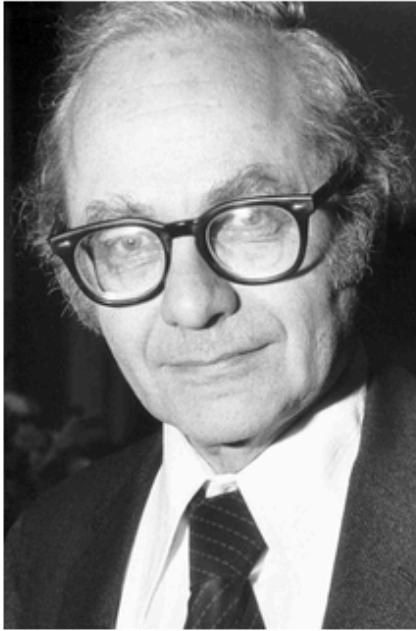


Photo from the Nobel Foundation archive.

David H. Hubel

Prize share: 1/4

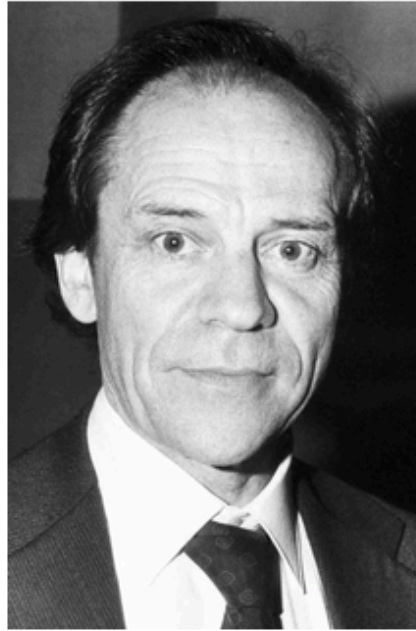


Photo from the Nobel Foundation archive.

Torsten N. Wiesel

Prize share: 1/4

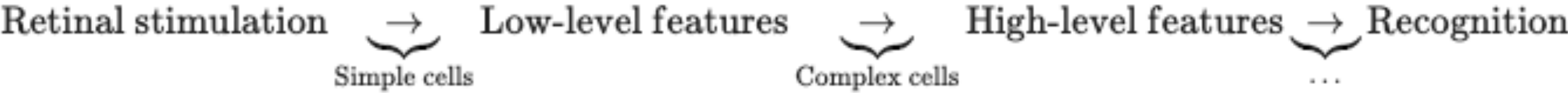


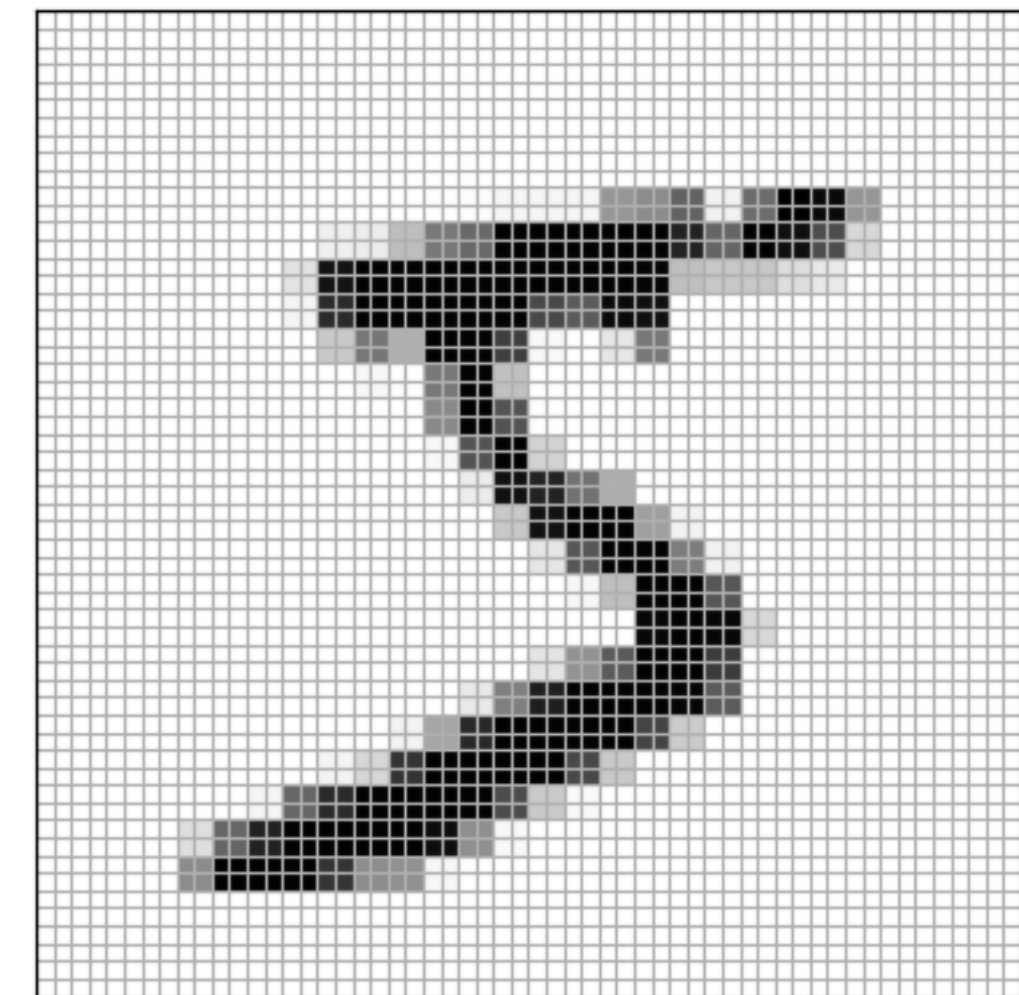
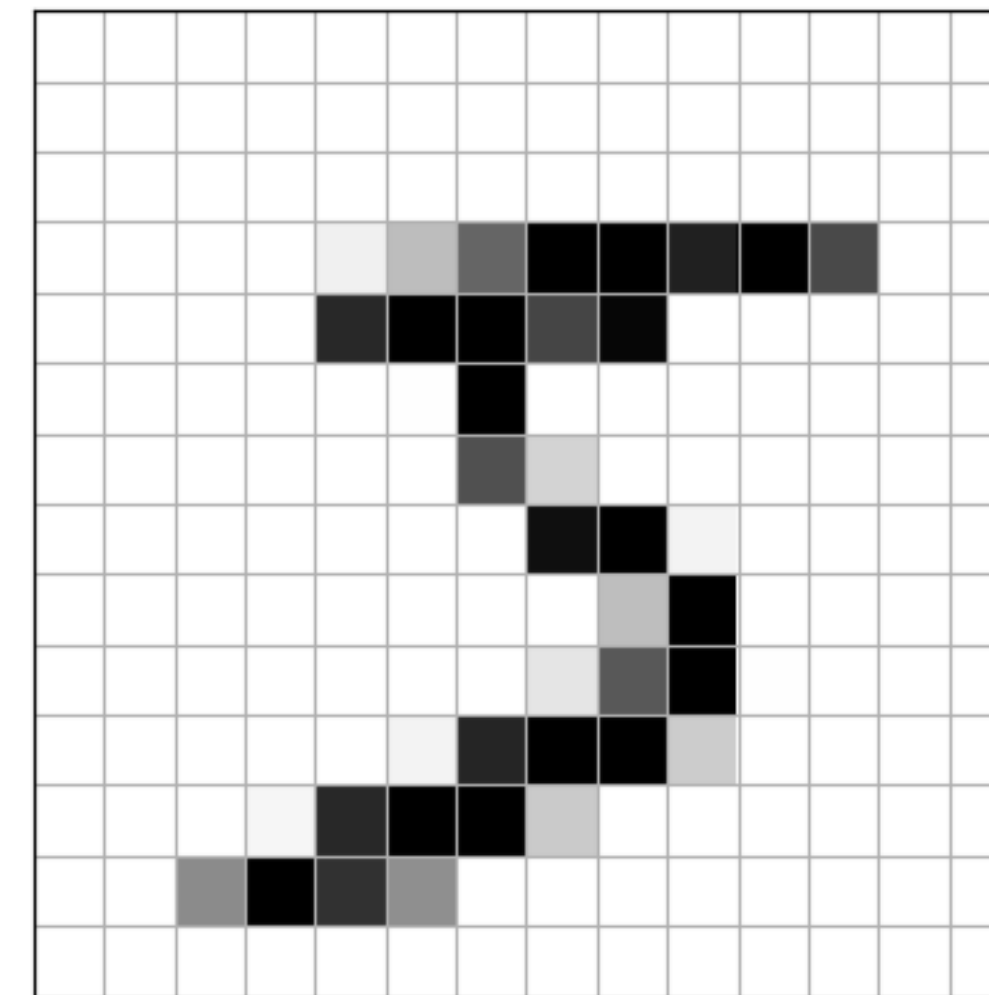
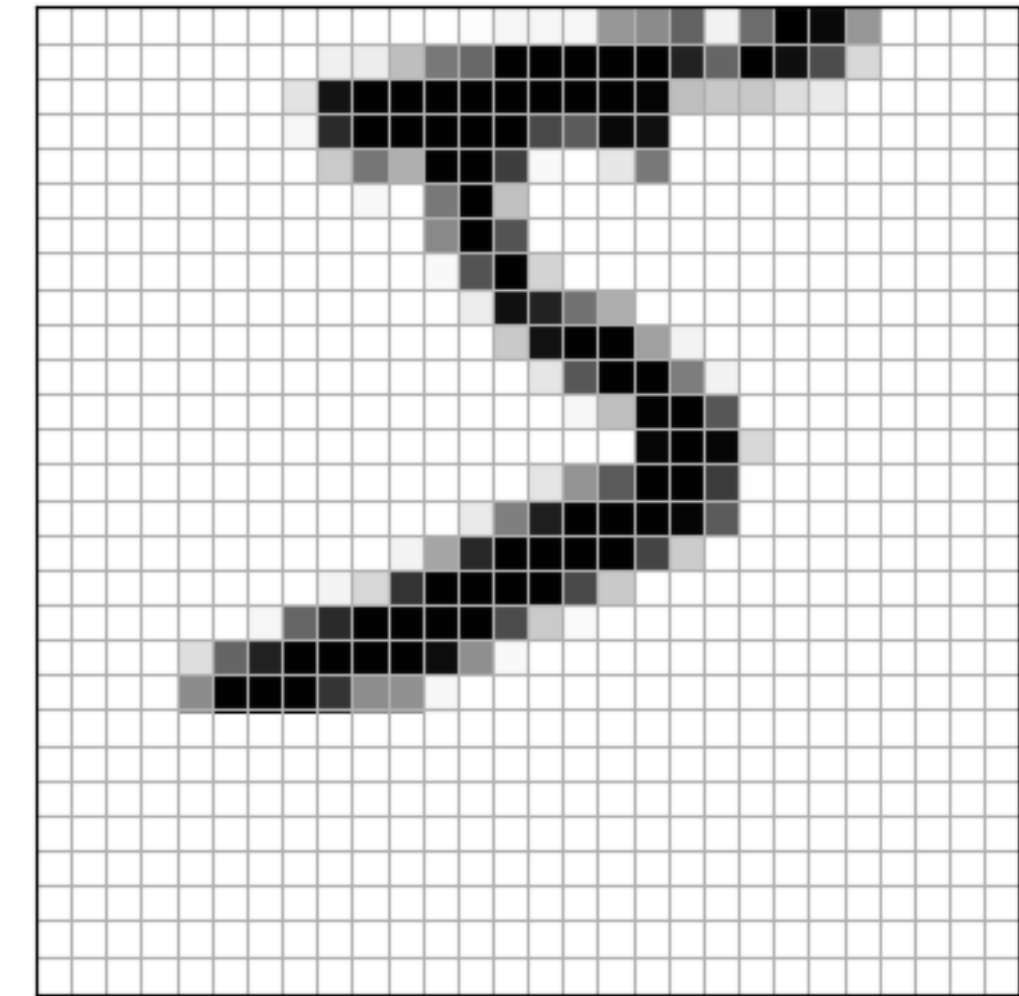
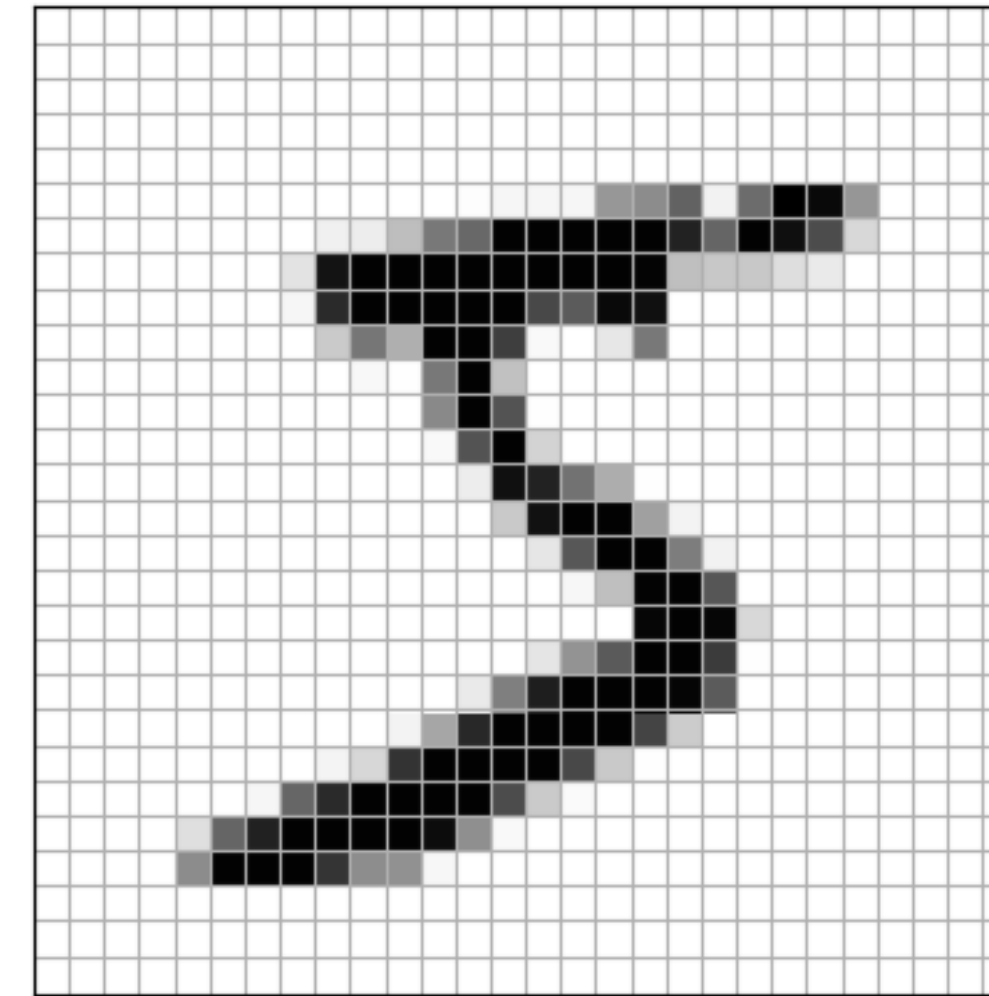
Image data

Image **data** is an array of pixel values

Image **features** are encoded within pixels

Q: How can we efficiently identify features within high-dimensional unstructured image data?

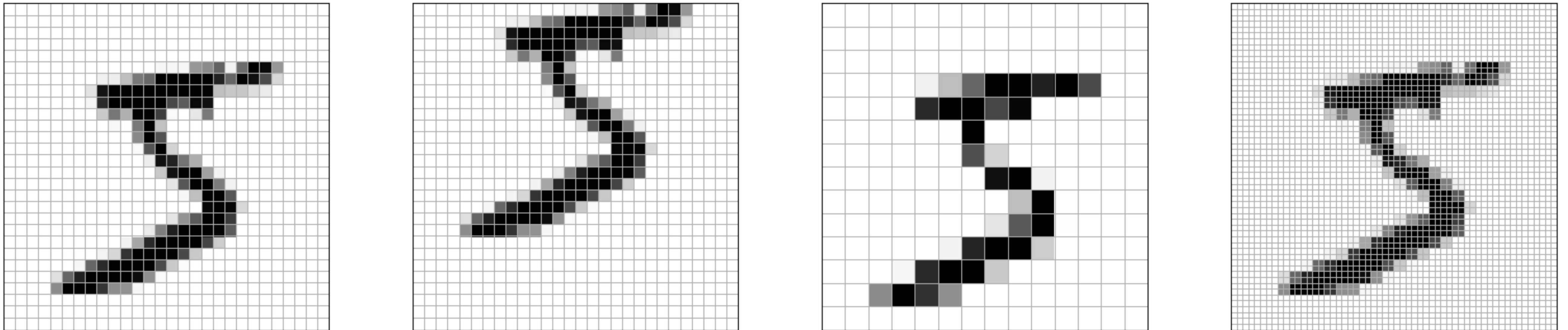
A: Exploit **grid geometry**, **sparse interactions**, and **parameter sharing**



MLP “flattened” geometry



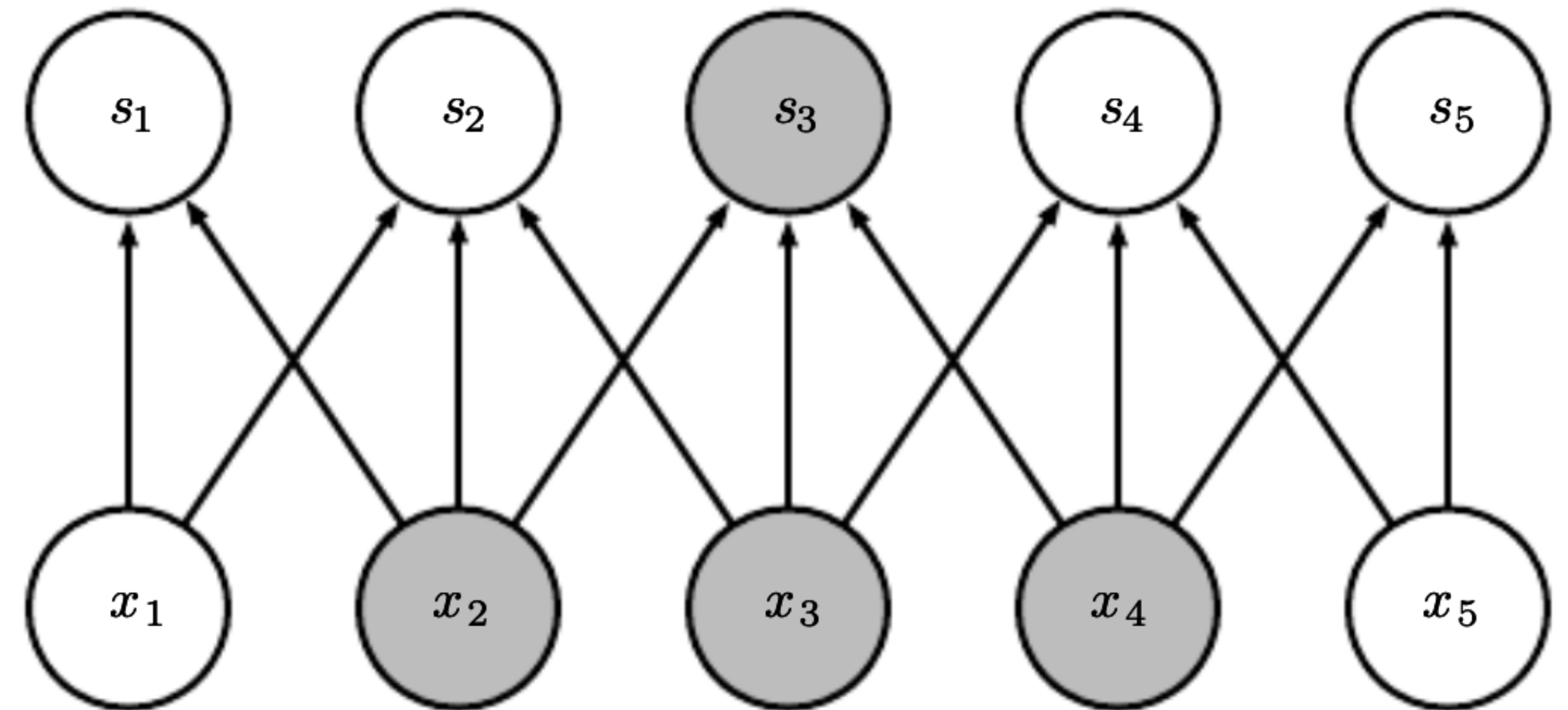
CNN “grid” geometry



Sparse connectivity

MLP (fully-connected) layers connect neurons in one layer with all others in a subsequent layer

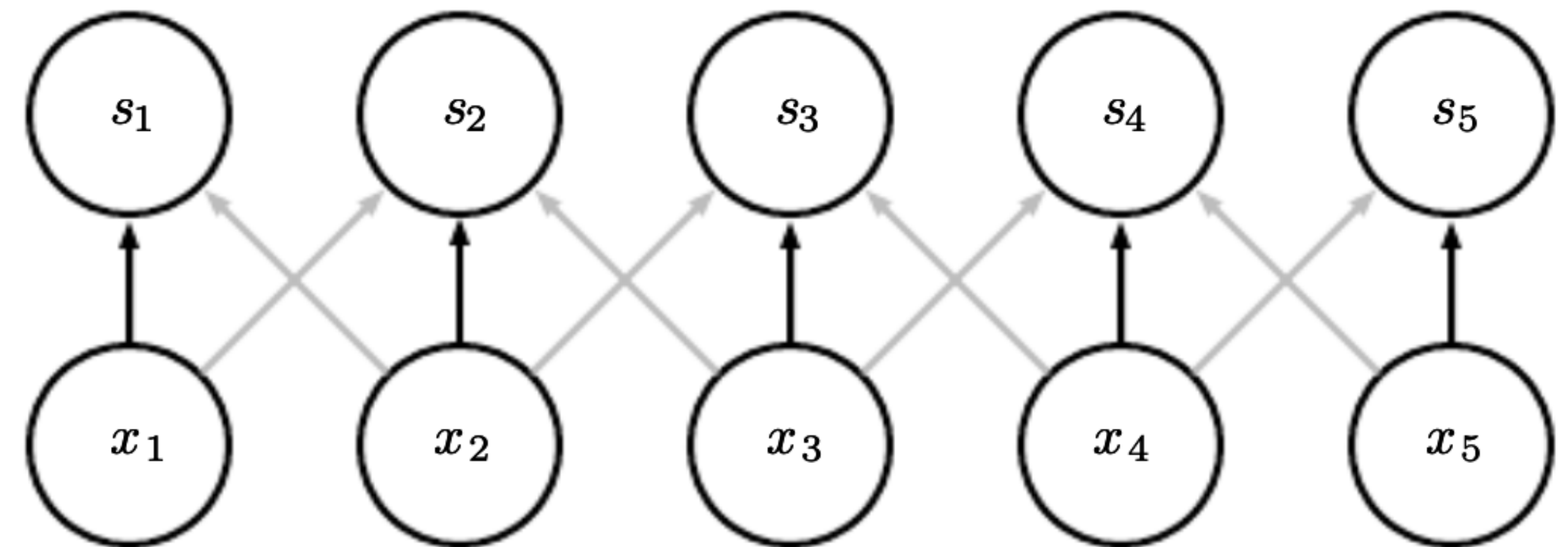
Sparse connections reduce the number of trainable weights, making computation more efficient



Parameter sharing

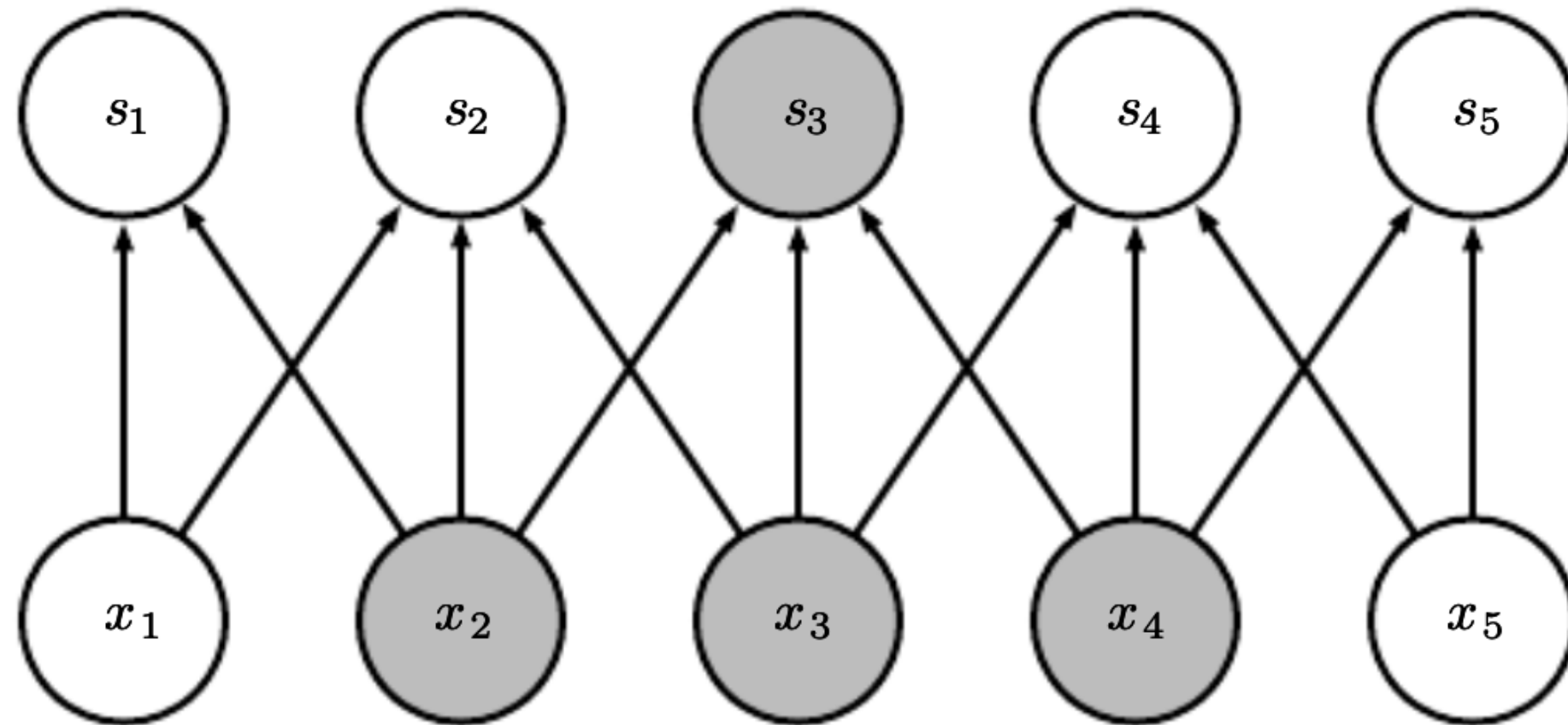
MLP (fully-connected) layers learn unique weights for each input-output pair

Parameter sharing uses the same weights at each input element, enabling feature identification *anywhere*



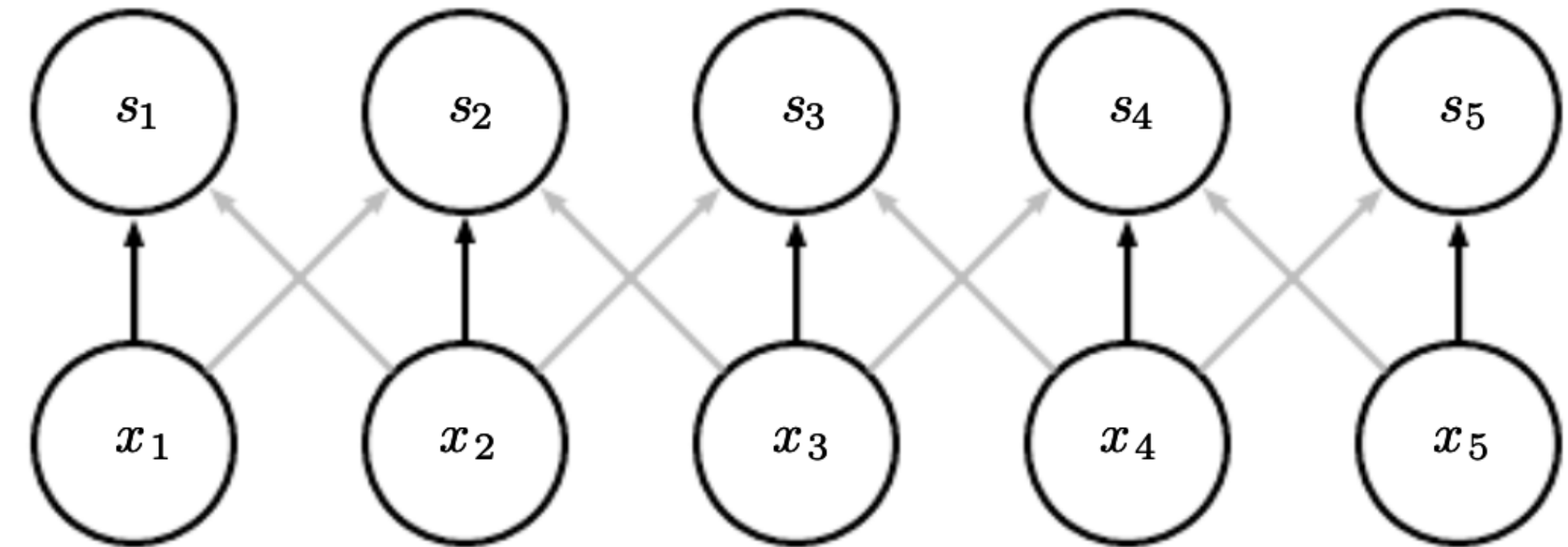
Putting things together

Sparse connectivity



+

Parameter sharing



$$f(g(x)) = g(f(x)) \quad f \text{ is equivariant to } g$$

This combination provides **translation equivariance**

Convolution

- The convolution of two functions $f(x)$, $w(x)$ is

$$(f \star w)(x) = \int_{-\infty}^{\infty} f(t) w(x - t) dt = \int_{-\infty}^{\infty} f(x - t) w(t) dt$$

- The discrete convolution is

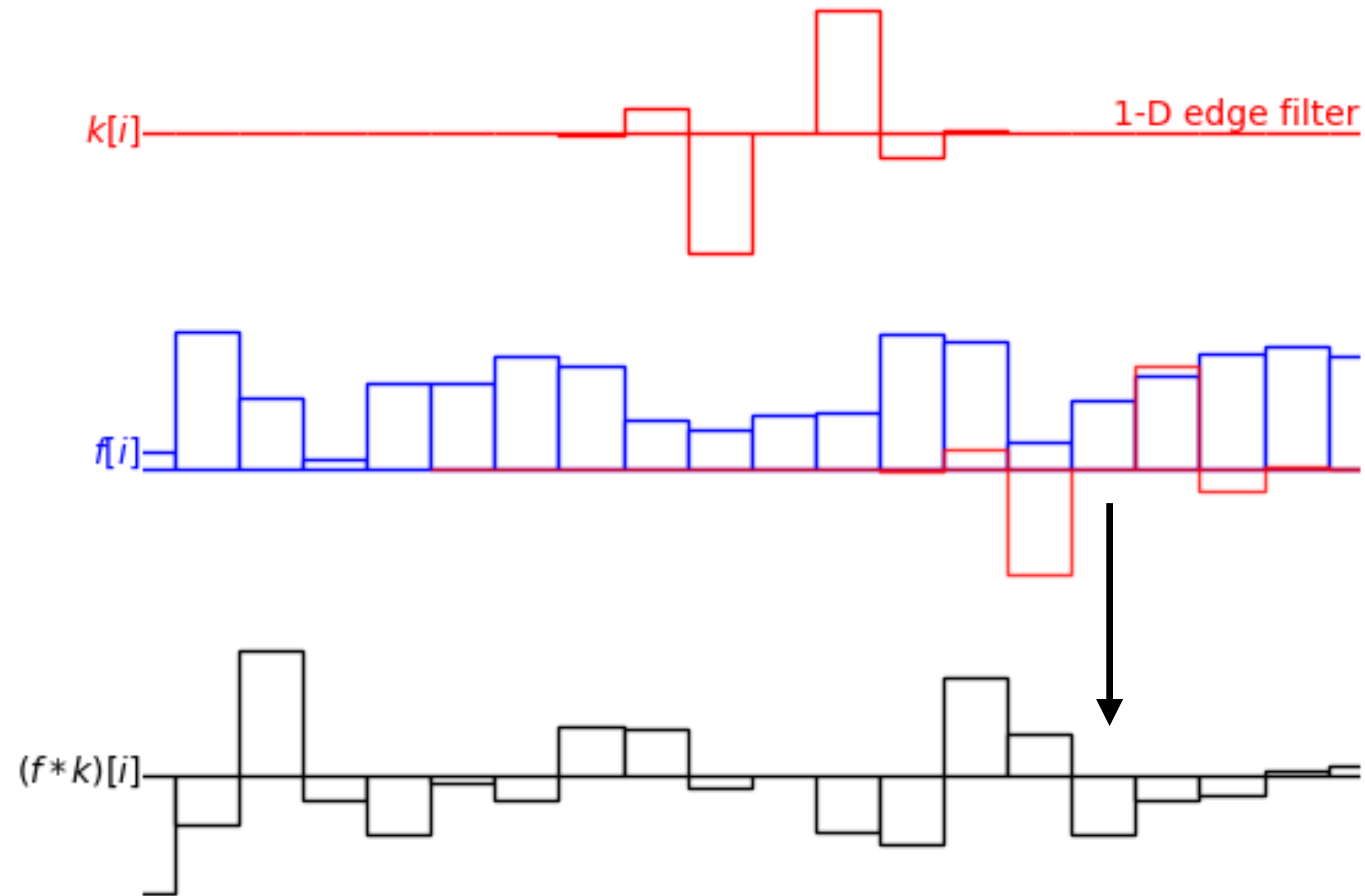
$$z[i] = \sum_{j=-\infty}^{\infty} x[i - j] w[j]$$

- For a kernel $w[j]$ of finite size (typical)

$$z[i] = \sum_{j=-K}^K x[i - j] w[j]$$

Convolution

$$(f \star k)[i] = \sum_{j=-K}^K f[i-j] k[j]$$



Convolution

1-D discrete convolution: $z[i] = \sum_{j=-K}^K x[i-j] w[j]$

2-D discrete convolution: $z[i,j] = \sum_{k=-K}^K \sum_{\ell=-K}^K x[i-k, j-\ell] w[k, \ell]$

3-D discrete convolution: $z[i,j,k] = \sum_{\ell=-K}^K \sum_{m=-K}^K \sum_{n=-K}^K x[i-\ell, j-m, k-n] w[\ell, m, n]$

Convolution

2-D discrete convolution:
$$z[i, j] = \sum_{k=-K}^K \sum_{\ell=-K}^K x[i - k, j - \ell] w[k, \ell]$$

- **Sparse connectivity:** kernel $w_{K \times K}$ may be smaller than input
- **Parameter sharing:** same kernel $w_{K \times K}$ is applied everywhere on the grid

Convolution in images

Input image

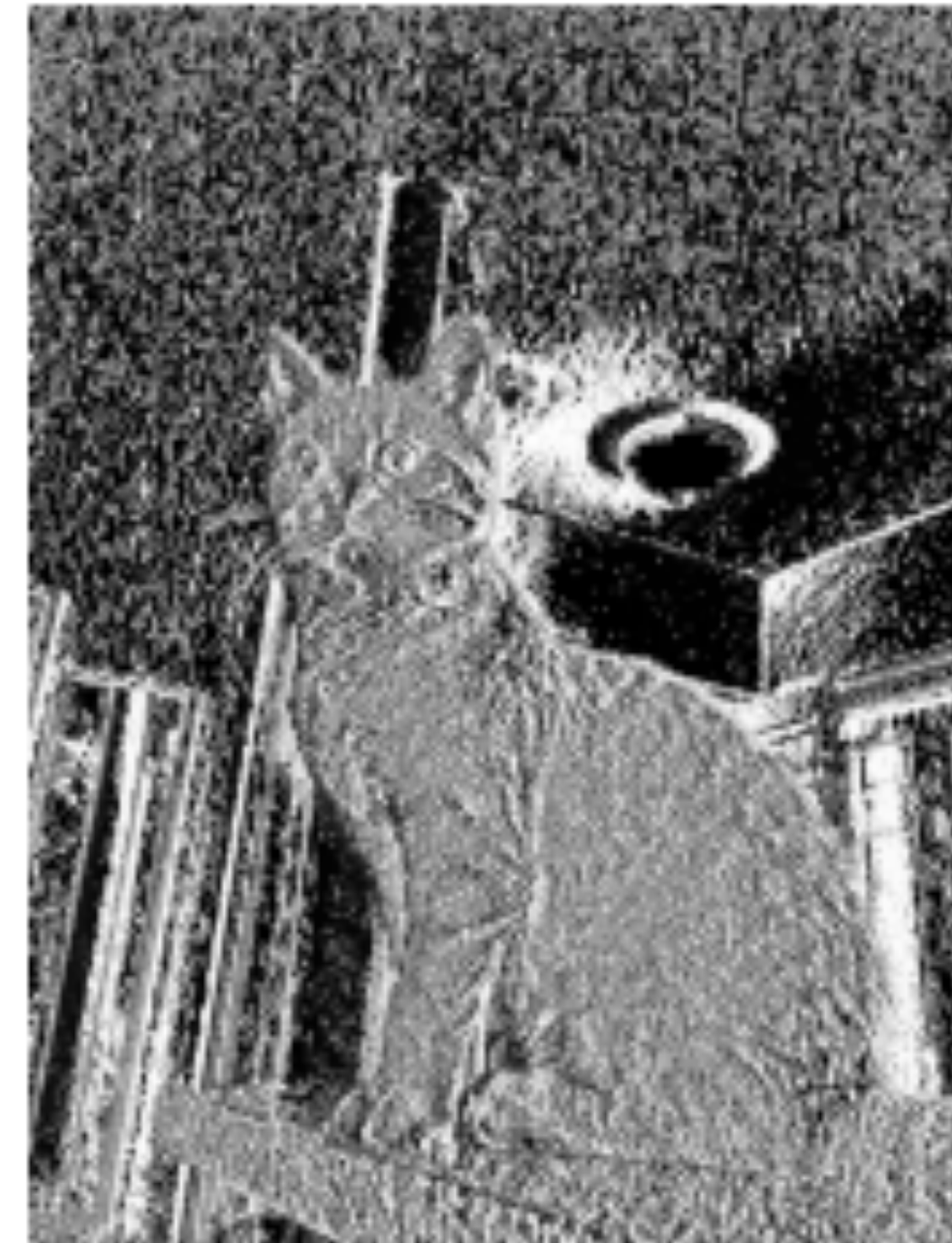


3x3 kernel

-1	0	1
-1	0	1
-1	0	1

(vertical edge filter)

Feature map



A **feature map** describes the *similarity* of a local pixel patch to some reference kernel

(convolution is equivalent to a sliding window dot product)

Convolution in images

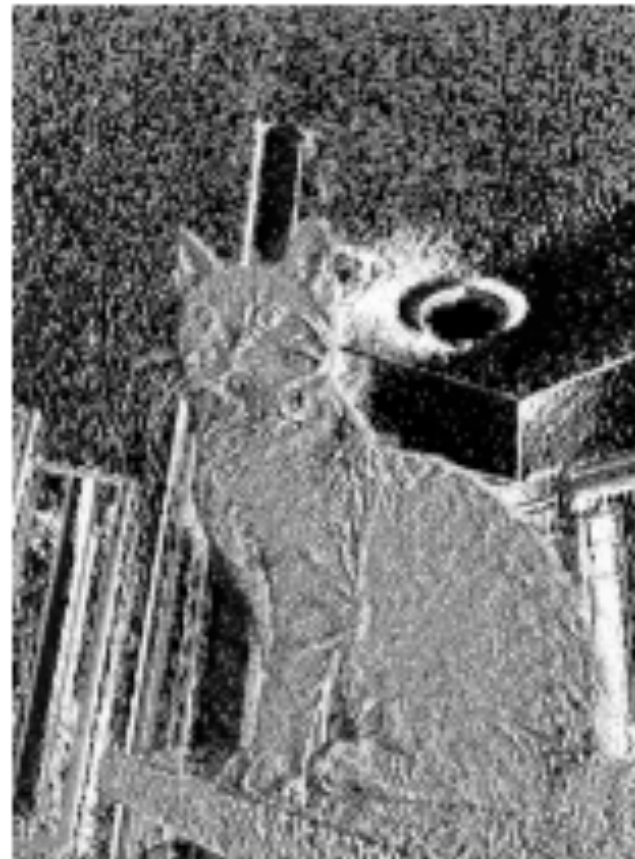
Input image



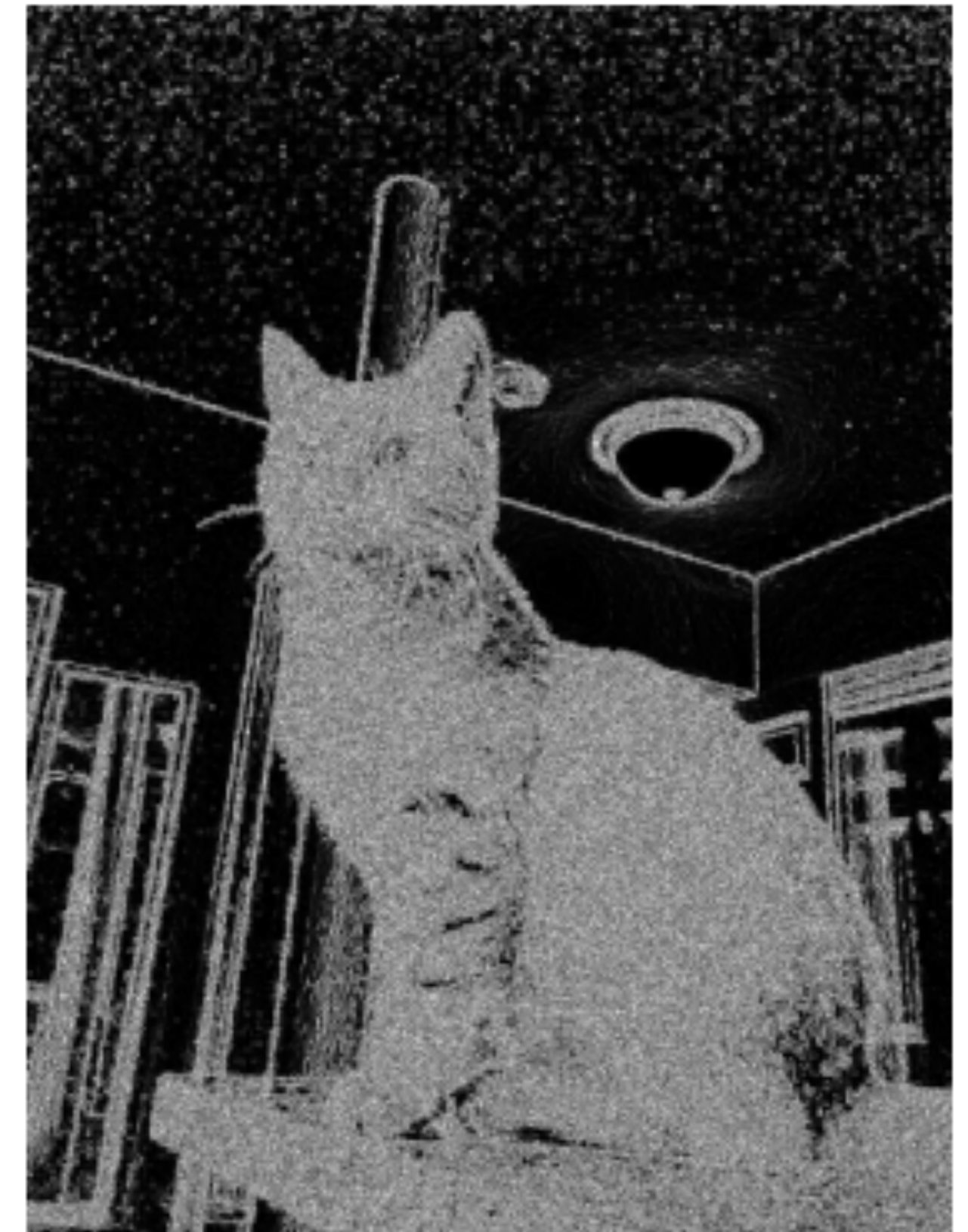
3x3 kernels

-1	0	1
-1	0	1
-1	0	1

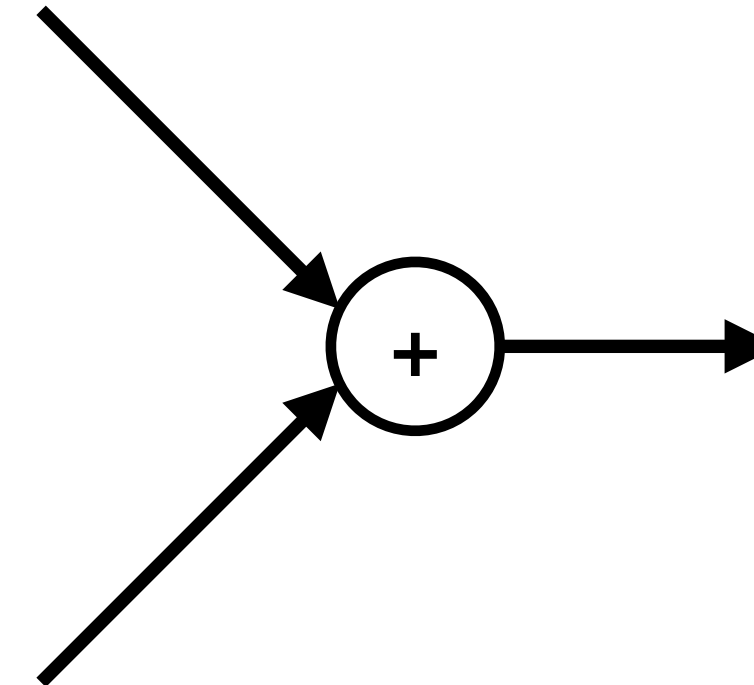
Feature maps



Output

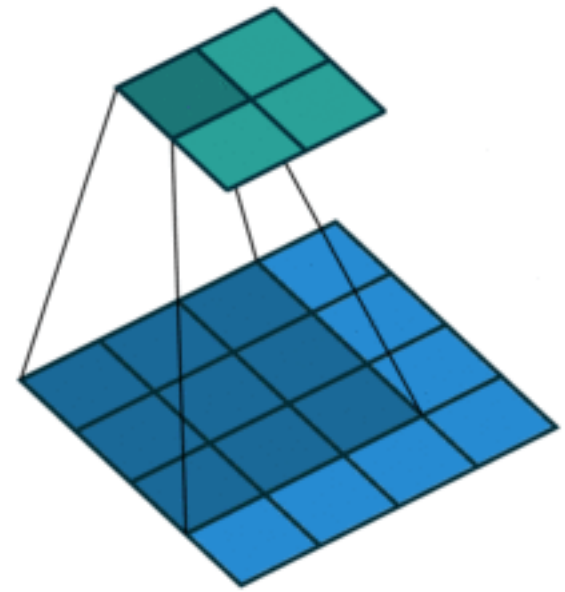


-1	-1	-1
0	0	0
1	1	1

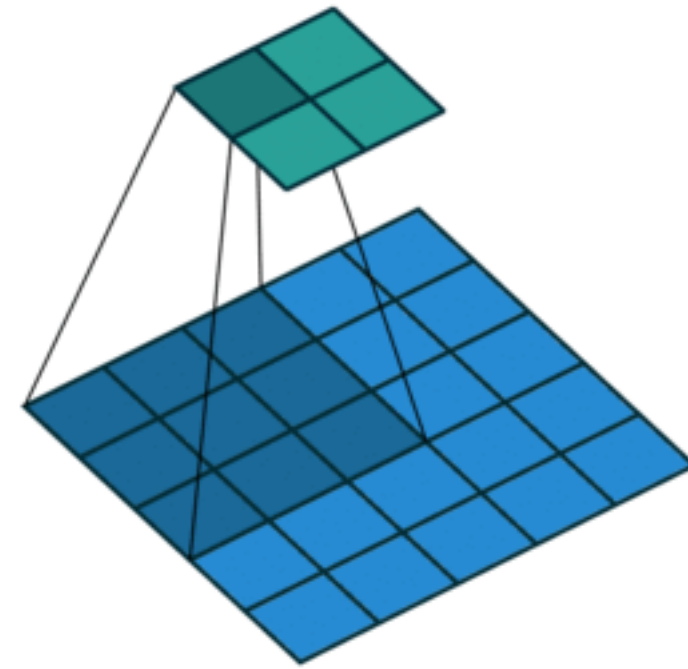


Other useful things to know

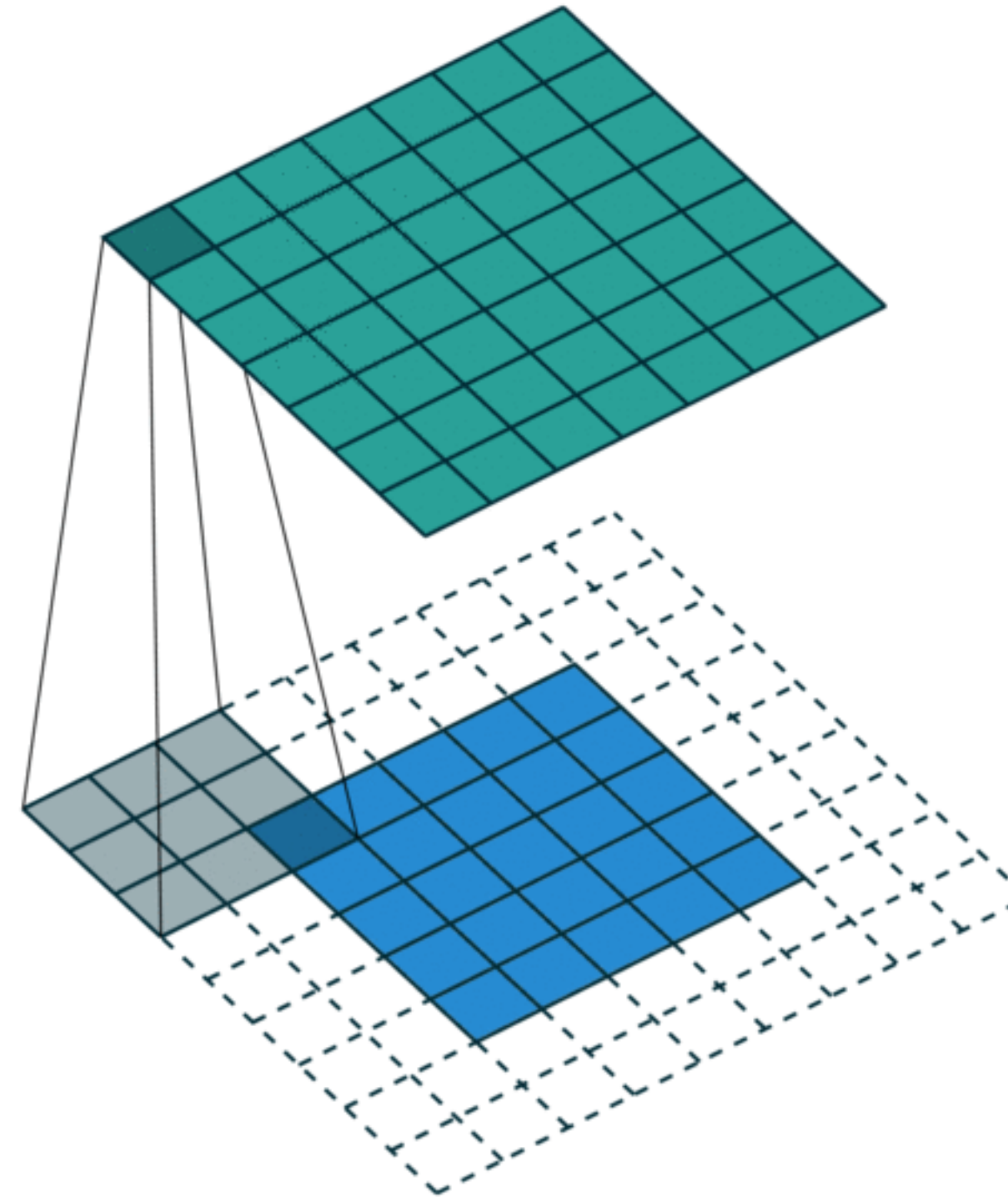
Convolution



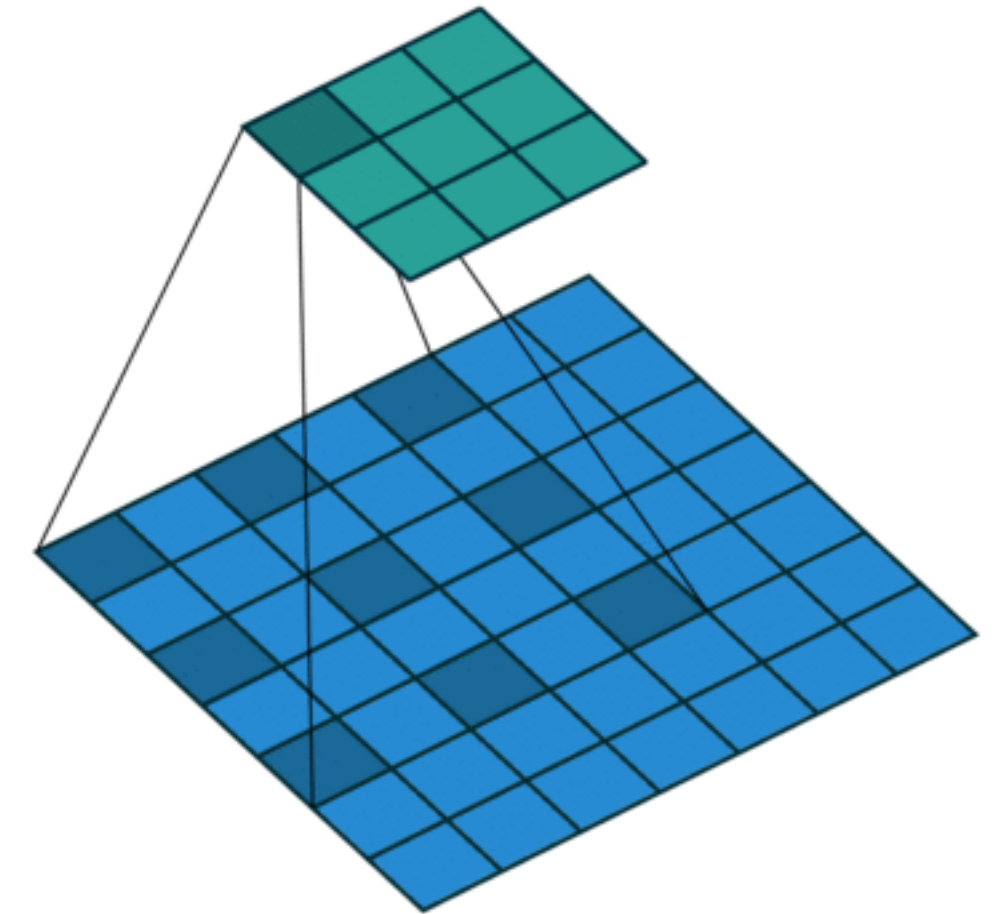
Strided Convolution



Padded Convolution



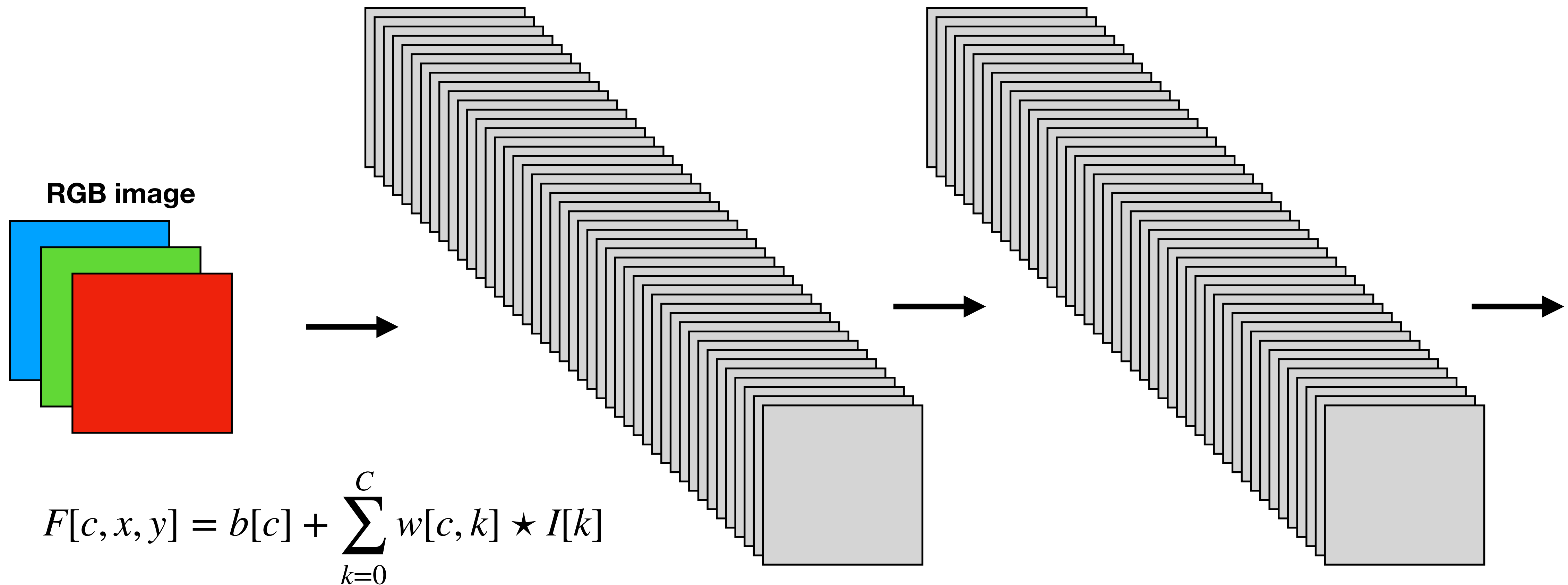
Dilated Convolution



See: https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md

The size of an output is $N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2 \times \text{pad} - \text{dilation} \times (\text{kernel size} - 1) - 1}{\text{stride}} + 1 \right\rfloor$

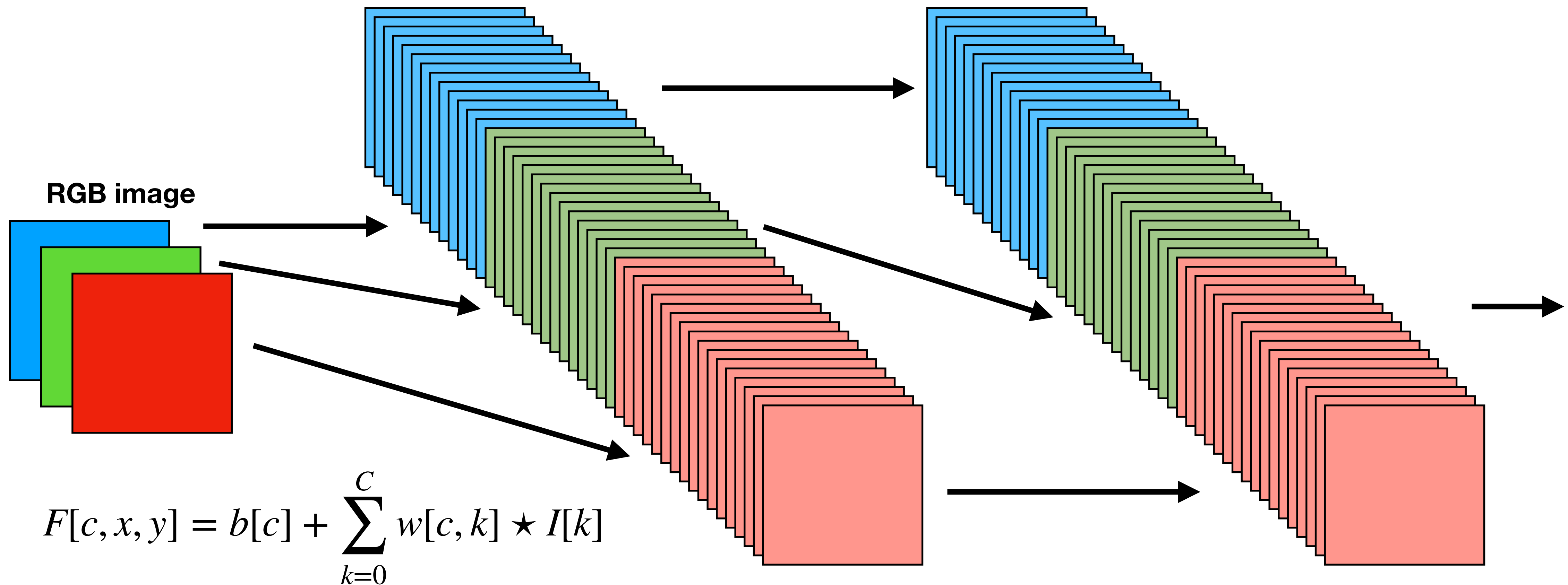
Adding another dimension: features



Convolutional layers learn **unique kernels** between each feature map

(**Parameter sharing:** a 1x1 kernel is the same as applying the same FC layer to each pixel)

Grouped convolution



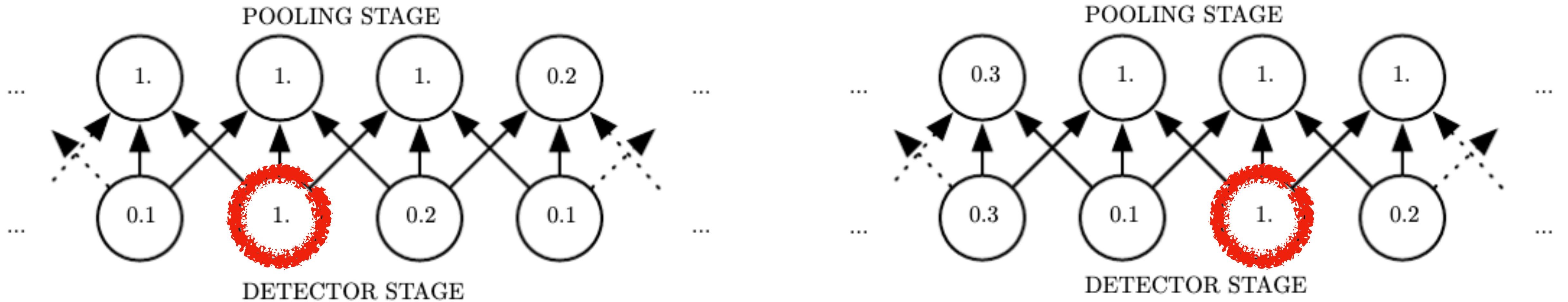
Grouped convolutions limit the connectivity between feature maps in each layer

(**sparse connectivity:** groups = C transforms each channel separately)

Why convolution?



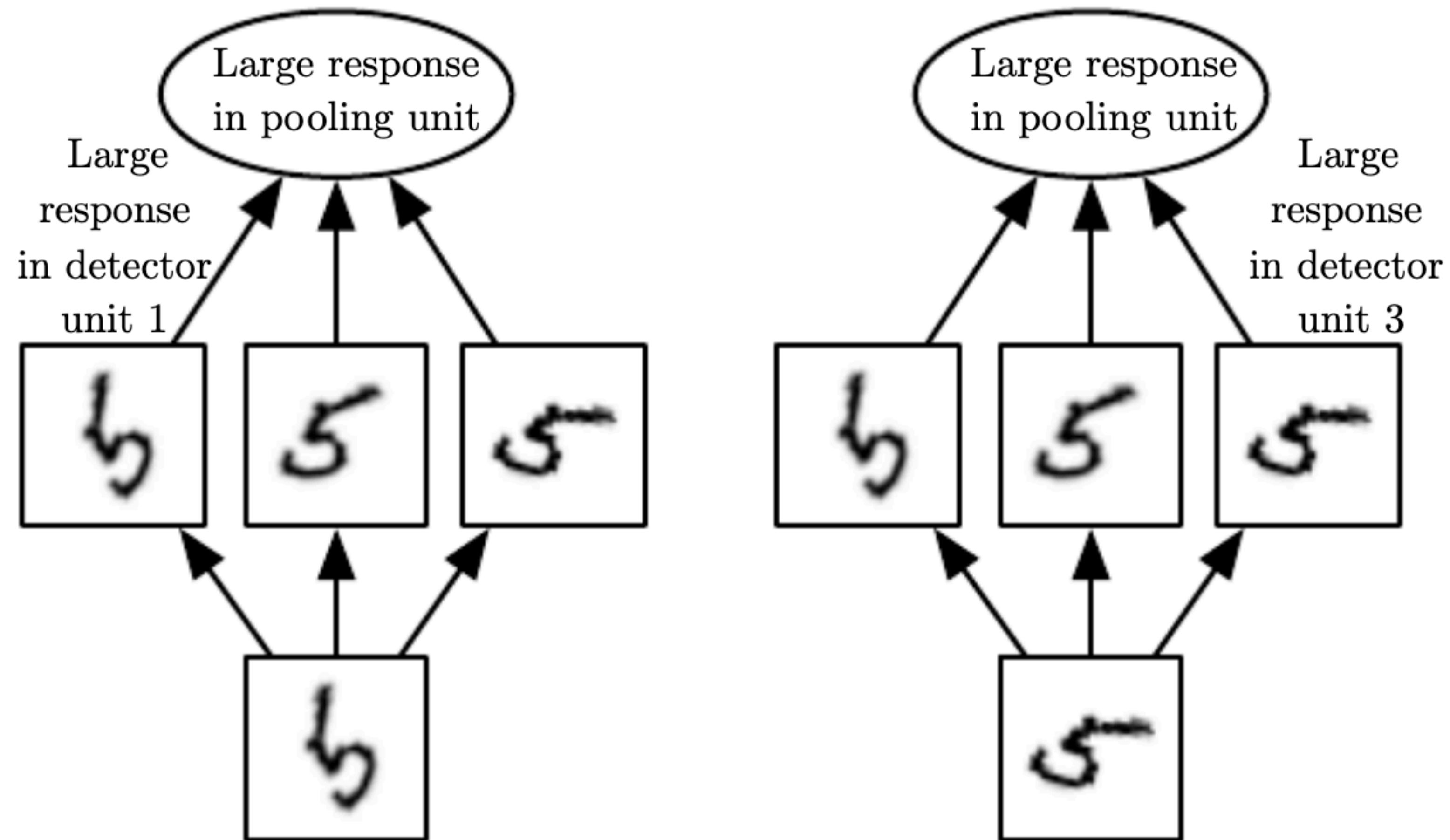
Local Maximum Pooling



Pooling allows a model to achieve translation **invariance**

→ Such models may detect a feature **anywhere**

Advantages of Max Pooling



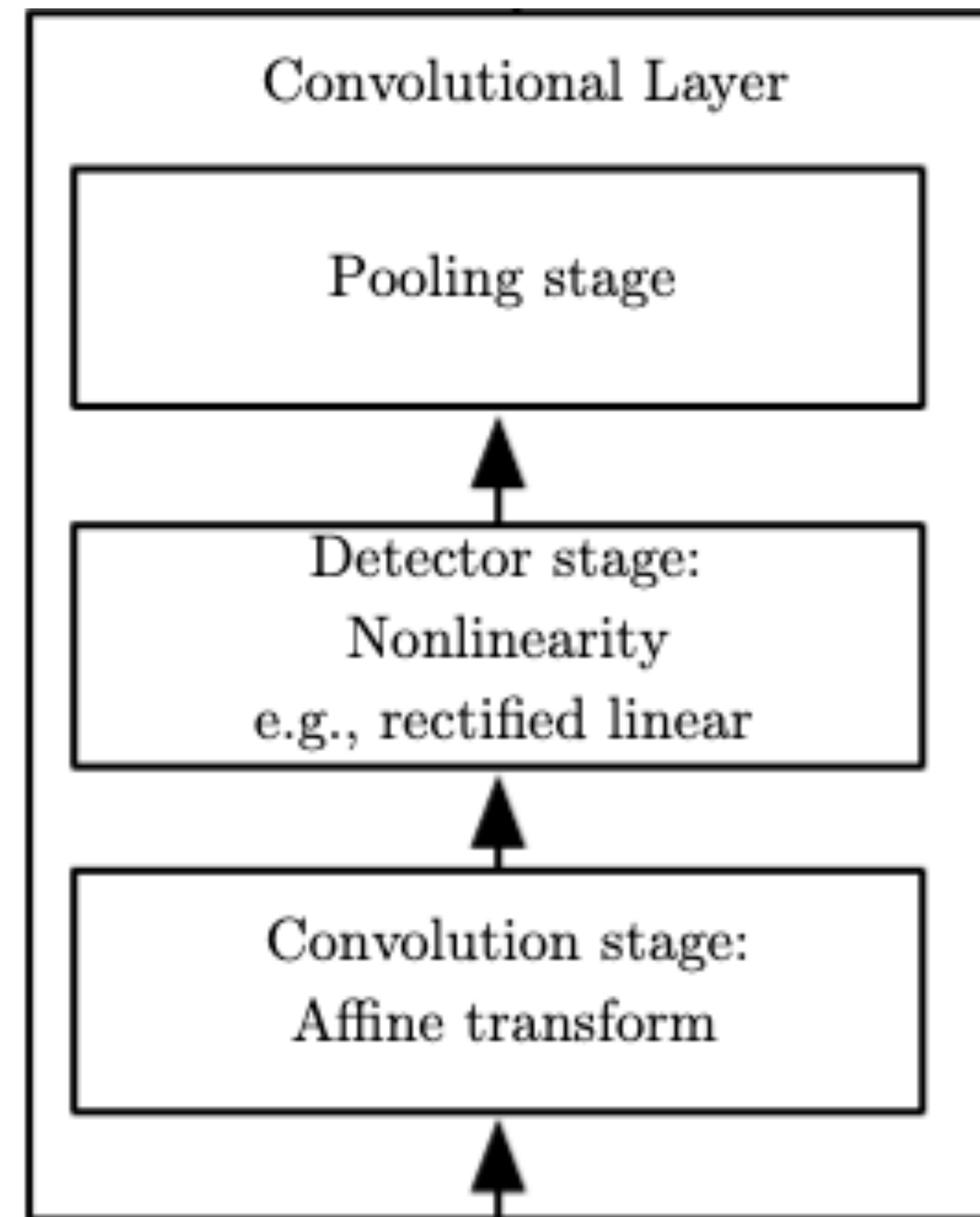
Pooling may enable **learned invariances**

Convolutional Neural Networks

Pooling for **translation invariance**

Nonlinearity for **expressivity**

Sparse connectivity + shared weights for **translation equivariance**



ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

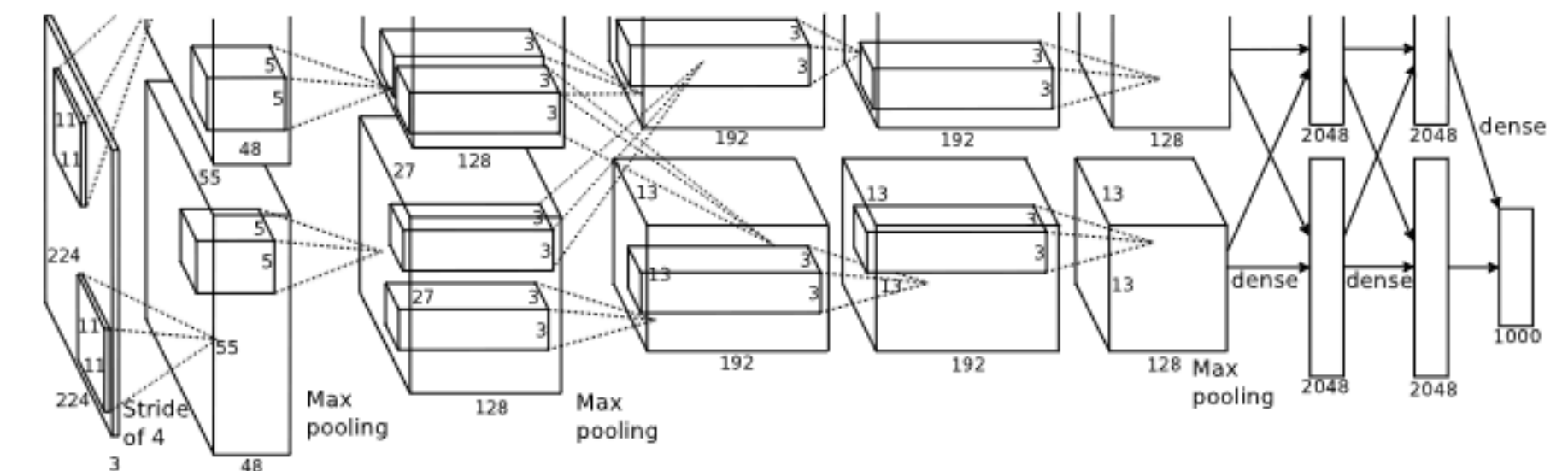


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Check out the code!

<https://github.com/computerhistory/AlexNet-Source-Code>

Hands-on with CNN



That's all!

- Slides will be posted on the course website: <https://colen-teaching.github.io/DATA442-Fall-2026>
- Lab demos will also be posted
- **PS1** due October 2